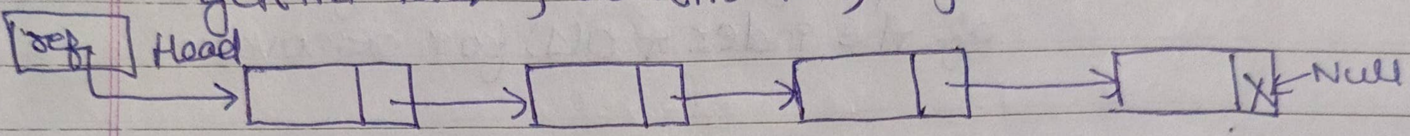
LinkedList

\# LinkedList : Data Structure used is Linked List.
- LinkedList implements both Deque & List Interface
- Means it support Deque Methods like :
  "getFirst", "getLast", "removeFirst", "removeLast" etc.

- It also support index based operation like list:
  "get(int idx)", "add(int idx, Object O)" etc.

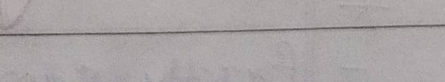


LinkedList

```
public class LinkedListExample{
    psvm(){
        LinkedList<Integer> list = new LinkedList<>();
        //using deque functionality.
        list.addLast(200);
        list.addLast(300);
        list.addLast(400);
        list.addFirst(100); // list = [100| ]->[200| ]->[300| ]->[400|x]
        Sout(list.getFirst()); // 100

        //using List functionality
        LinkedList<Integer> list2 = new LinkedList<>();
        list2.add(0, 100);
        list2.add(1, 300);
        list2.add(2, 400); // list2 =
        list2.add(1, 200);

        Sout(list2.get(1) + " " + list2.get(2));
    }
}
```



output: 100
~~200~~ 300

Page No :-
Date :- / /

Time complexity:
- Insertion at start and end : O(1)
- Insertion at particular index : O(n) for lookup of the index + O(1) for adding
- Search : O(n) // linear search only possible
- Deletion at start or end : O(1)
- Deletion at specific index : O(n) for the lookup of the index + O(1) for removal

Space complexity : O(n)

Collection Till Now	isthreadSafe	Maintains insertion order	Null Elements allowed	Duplicate ele allowed
LinkedList	No	Yes	Yes	Yes
Vector	Yes	Yes	Yes	Yes
Stack	Yes	No	Yes	Yes

ThreadSafe varient of linkedlist : CopyOnWriteArraylist

* Vector

- Exactly same as arraylist, elements can be accessed via index.
- Vector is threadsafe as its all methods are synchronized by default.
- Puts lock when operation is performed on Vector
- Less efficient than arraylist as for each operation it do lock/unlock internally.

```
public class VectorExample{
    psvm(){
        Vector<Integer> obj = new Vector<>();
        obj.add(0, 200);
        obj.add(1, 300);
        Sout(obj.get(0)); // 200
    }
}
```

STACK

- Represent LIFO (last in First out) operation
- Since it extends vector, its method is also Synchronized.

Q How Stack is different from deque?
→ Deque is not threadsafe whereas Stack is threadsafe

```
public class StackExample{
    psvm(){
        Stack<Integer> stack = new Stack<>();
        stack.push(10);
        stack.push(20);      // stack = | 20 | ← Top of stack
                             //         | 10 |
        stack.pop();         //  20 popped, | 10 |
        Sout(stack.peek()); // 10
    }
}
```

Methods of Stacks are :	Time Complexity
Push()	O(1)
Pop()	O(1)
peek()	O(1)

Space complexity : O(n)